

【数字游戏】

题目描述

小明玩一个游戏。

系统发 $1+n$ 张牌，每张牌上有一个整数。

第一张给小明，后 n 张按照发牌顺序排成连续的一行。

需要小明判断，后 n 张牌中，**是否存在** 连续的若干张牌，其和可以整除小明手中牌上的数字。

输入描述

输入数据有多组，每组输入数据有两行，输入到文件结尾结束。

第一行有两个整数 n 和 m ，空格隔开。 m 代表发给小明牌上的数字。

第二行有 n 个数，代表后续发的 n 张牌上的数字，以空格隔开。

输出描述

对每组输入，如果存在满足条件的连续若干张牌，则输出1;否则，输出0

备注

- $1 \leq n \leq 1000$
- $1 \leq \text{牌上的整数} \leq 400000$
- 输入的组数，不多于1000
- 用例确保输入都正确，不需要考虑非法情况。

用例

输入	6 7 2 12 6 3 5 5 10 11 1 1 1 1 1 1 1 1 1 1
输出	1 0
说明	两组输入。第一组小明牌的数字为7，再发了6张牌。第1、2两张牌数字和为14，可以整除7，输出1，第二组小明牌的数字为11，再发了10张牌，这10张牌数字和为10，无法整除11，输出0。

题目解析

本题考察前缀和知识，以及 数学问题 。

本题要求连续若干张牌的和，其实就是求区间的元素之和。

而求解任意区间和的最佳算法就是：前缀和之差，这个知识点可以看下

利用前缀和，我们可以在O(1)的时间求解出[L,R]区间和 = preSum[R] - preSum[L-1]。

那么接下来，我们是否需要双重for遍历出所有区间，然后逐一验证对应区间和是否可以整除k呢？

本题中每组用例最多n = 1000张牌，因此双重for遍历对应牌的所有连续区间的时间复杂度O(n^2)。

而本题有最多有1000组用例，因此相当于O(n^3)的时间复杂度，这是非常容易超时的。

本题优化是基于一个数学性质：

如果两个数a,b，他们的差a - b可以被k整除，则必然 $a \% k == b \% k$ 。
反之，如果 $a \% k == b \% k$ ，那么 a - b 必然可以被k整除。

大家可以自行验证下这个数学性质。

验证好后，我们只需要将preSum[R]带入a，preSum[L-1]带入b 再来看看：

如果 $\text{preSum}[R] \% k == \text{preSum}[L-1] \% k$ ，那么必然 $\text{preSum}[R] - \text{preSum}[L-1]$ 可以被k整除。

利用这个数学性质，我们就不必双重for遍历所有区间了，只需要检查preSum数组中是否存在两个元素 $\% k$ 的结果相同，即可说明存在区间之和可以整除k。

2024.9.11

本题涉及不定行输入解析，这里补充一下不定行输入解析的知识

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6    while ((line = await readline())) {
7      const [n, m] = line.split(" ").map(Number);
8      const nums = (await readline()).split(" ").map(Number);
9      console.log(isExsit(nums, m));
10   }
11 })();
12
13 function isExsit(nums, m) {
14   const remain = new Set();
15   remain.add(0);
16
17   let sum = 0;
18   for (let num of nums) {
19     sum += num;
20     if (remain.has(sum % m)) {
21       return 1;
```

运行

```

22     } else {
23         remain.add(sum % m);
24     }
25 }
26
27 return 0;
28 }

```

Java算法源码

```

1  import java.util.Arrays;
2  import java.util.HashSet;
3  import java.util.Scanner;
4
5  public class Main {
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8
9          while (sc.hasNextLine()) {
10             int[] tmp = Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
11             int[] nums = Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
12             System.out.println(isExist(nums, tmp[1]));
13         }
14     }
15
16     public static int isExist(int[] nums, int m) {
17         HashSet<Integer> remain = new HashSet<>();
18         remain.add(0);
19
20         int sum = 0;
21         for (int num : nums) {
22             sum += num;
23             if (remain.contains(sum % m)) {
24                 return 1;

```

```

25         } else {26         remain.add(sum % m);
27     }
28 }
29
30     return 0;
31 }
32 }

```

Python算法源码

```

1  def isExist(nums, m):
2      remain = set()
3      remain.add(0)
4
5      total = 0
6      for num in nums:
7          total += num
8          if total % m in remain:
9              return 1
10         else:
11             remain.add(total % m)
12
13     return 0
14
15
16 # 输入获取
17 while True:
18     try:
19         n, m = map(int, input().split())
20         nums = list(map(int, input().split()))
21         print(isExist(nums, m))
22     except:
23         break

```

C算法源码

```
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  #define MAX_NUM 400001
5
6  int isExist(const int nums[], int nums_size, int m) {
7      bool occurs[MAX_NUM] = {0};
8      occurs[0] = true;
9
10     int sum = 0;
11     for (int i = 0; i < nums_size; i++) {
12         int num = nums[i];
13
14         sum += num;
15
16         if (occurs[sum % m]) {
17             return 1;
18         } else {
19             occurs[sum % m] = true;
20         }
21     }
22
23     return 0;
24 }
25
26 int main() {
27     int n, m;
28     while (scanf("%d %d", &n, &m) != EOF) {
29         int nums[n];
30         for (int i = 0; i < n; i++) {
31             scanf("%d", &nums[i]);
32         }
33
34         printf("%d\n", isExist(nums, n, m));
35     }
```

```
36 |  
    37 |         return 0;  
38 | }
```

C++ 算法源码

```
1  #include <bits/stdc++.h>  
2  
3  using namespace std;  
4  
5  int isExist(vector<int> &nums, int m) {  
6      set<int> remain;  
7      remain.insert(0);  
8  
9      int sum = 0;  
10     for (const auto &num: nums) {  
11         sum += num;  
12  
13         if (remain.count(sum % m) > 0) {  
14             return 1;  
15         } else {  
16             remain.insert(sum % m);  
17         }  
18     }  
19  
20     return 0;  
21 }  
22  
23 int main() {  
24     int n, m;  
25     while (cin >> n >> m) {  
26         vector<int> nums(n);  
27         for (int i = 0; i < n; i++) {  
28             cin >> nums[i];  
29         }
```

```
30 |  
31 |         cout << isExist(nums, m) << endl;  
32 |     }  
33 |  
34 |     return 0;  
35 | }
```